

Validate Your Memory System

Prove the files you built this week are real, correct, and working together.

You've already built the pieces. In this week's exercises, you had Claude Code create your `memory.md` and `decision.md`, set up your `structure.md`, and install **Beads** (`.beads`). **This assignment does not ask you to make anything new** — it validates that those files exist, meet the standard, and actually work as a memory system.

 **Outcome-based & evidence-based.** Your **GitHub repository is the evidence** — it already contains the files you made in the exercises. Here you run a validation pass to confirm each one meets the bar, and prove the system works end-to-end.

STEP 1 — SELF-AUDIT

Tally your files against the standard

Open each file you created in the exercises and confirm it meets the bar below. Tick each row only if it genuinely passes.

File (from your exercises)	Standard it must meet	Passes?
<code>memory.md</code>	5+ dated entries — each a real task + one true lesson (not filler)	☐
<code>decision.md</code>	3+ decisions — each with what · why · what you ruled out	☐
<code>structure.md</code>	maps your folders, and you can show the agent finding one file	☐
<code>.beads</code>	5+ tracked tasks across at least two states (e.g. in progress, done)	☐

STEP 2 — LET CLAUDE AUDIT IT

Have Claude Code check your work

In your project, ask Claude Code to audit the files and report back. Paste this:

```
"Audit my project's memory files and tell me, for each, whether it meets the standard:
• memory.md — 5+ dated entries, each a real lesson
• decision.md — 3+ entries with what / why / ruled-out
• structure.md — maps the folders
• .beads — 5+ tasks across at least two states.
List anything missing or weak."
```

If Claude flags a gap, fix it the same way you did in the exercises (ask Claude to revise), then re-run the audit until everything passes. **You're verifying — not rewriting from scratch.**

STEP 3 — CAPSTONE · PROVE IT WORKS

Pick the project up cold in a fresh chat

Open your project in a **brand-new chat** with your agent. Without re-explaining anything, ask it to use its memory files to answer:

```
"Using only the project's files, tell me: the rules you follow, my most recent lesson, one key decision and why, where the data lives, and what's left to do."
```

This is the real test. If the agent answers correctly — pulling from `CLAUDE.md`, `memory.md`, `decision.md`, `structure.md`, and `.beads` — your memory system works. If it can't, a file is missing or thin: go back and strengthen it.

Confirm everything is on GitHub

Make sure all the files are pushed to your existing project's GitHub repository — the same project repo you've used all week, **not a new one**. That repo is your submission. From your project folder:

```
git add memory.md decision.md structure.md .beads CLAUDE.md
git commit -m "Memory Architecture: validated memory system"
git push
```

(Most of these were pushed during the exercises — this just confirms the latest versions are up.)

✦ **Submit:** share the link to your GitHub repository. You pass the assignment when the repo contains all four files meeting the standard in Step 1, **and** your agent passes the Step 3 cold-start test.